



UNIVERSITÉ SAINT JEAN
SAINT JEAN INGÉNIEUR



► Cahier de Réalisation : SOFTWARE DEFINED NETWORK AVEC NEUTRON

Présenté par :

- KAPTUE KEMGNE YANN MAEL
- TCHINWA ISMAEL CLINTON

Année Académique
2025-2026

TABLE OF CONTENTS

01

Introduction

02

Analyse des besoins

03

Choix technologiques

04

Architecture technique

05

Phase de réalisation

06

Contraintes rencontrés

07

**Conclusion et
perspectives**



► Introduction

01

► 1.1 Contexte

Avec l'évolution des infrastructures cloud et des architectures virtualisées, les réseaux traditionnels montrent des limites en matière d'agilité, d'automatisation et d'isolation multi-tenant. Les entreprises modernes ont besoin de réseaux dynamiques capables d'être provisionnés à la demande tout en garantissant sécurité, segmentation et haute disponibilité.

La solution **Software Defined Networking (SDN)** permet de séparer le plan de contrôle du plan de données afin d'automatiser et centraliser la gestion réseau. Dans les environnements cloud, cette approche est implémentée notamment via **OpenStack** et son composant réseau **OpenStack Neutron**.



► 1.2 Problématique

Comment concevoir et déployer une architecture réseau virtualisée multi-tenant permettant :

- L'isolation stricte entre locataires (tenants),
- La gestion centralisée du réseau,
- La création dynamique de réseaux virtuels,
- L'exposition d'API sécurisées,
- La simulation d'un environnement cloud réaliste ?

► 1.3 Objectifs du projet

Objectif général

Mettre en place une infrastructure SDN multi-tenant basée sur OpenStack intégrant Neutron pour la virtualisation réseau.

Objectifs spécifiques

- ★ Déployer un environnement OpenStack via DevStack
- ★ Implémenter une segmentation réseau (VLAN/VXLAN)
- ★ Créer trois tenants isolés (Tenant A, Tenant B, Tenant C)
- ★ Configurer routeurs virtuels, réseaux privés et réseaux externes
- ★ Exposer et tester les API OpenStack
- ★ Développer une interface graphique simplifiée de gestion

► **Analyse des
besoins**

02

► 2.1 Besoins fonctionnels



Le système doit permettre :

- **Création de projets (tenants)**
- **Création de réseaux privés virtuels**
- **Création de routeurs virtuels**
- **Association Floating IP** : consiste à lier une adresse IP publique du réseau externe à l'adresse IP privée d'une machine virtuelle afin de permettre son accès depuis Internet.
- **Isolation inter-tenant** : les réseaux virtuels de différents projets ne peuvent communiquer entre eux sans configuration explicite du routage ou règles spécifiques.
- **Communication intra-tenant** : la capacités des VMs appartenant au même projet de communiquer librement entre eux via leur réseau privé virtuel.
- **Supervision via interface graphique**

► 2.2 Besoins non fonctionnels

Sécurité

- Isolation stricte via VXLAN/VLAN
- Sécurisation API Keystone
- Gestion RBAC
- Filtrage via Security Groups

Performance

- Latence minimale intra-tenant
- Scalabilité horizontale
- Capacité à supporter plusieurs VM simultanées

Fiabilité

- Haute disponibilité des services OpenStack
- Persistance des configurations

Évolutivité

- Ajout futur de compute nodes pour gérer l'exécution des VMs
- Extension vers architecture multi-node



► **Choix technologiques**

03

► 3.1 Plateforme Cloud

OpenStack : Plateforme IaaS open-source permettant la gestion complète d'un cloud privé.

Modules utilisés :

- **Keystone (Identity)** : gère l'authentification des utilisateurs ainsi que les rôles et permissions d'accès aux services OpenStack.
- **Glance (Images)** : stocke et fournit les images de systèmes d'exploitation utilisées pour créer les machines virtuelles.
- **Nova (Compute)** : responsable de la création, de la gestion et de l'exécution des machines virtuelles.
- **Neutron (Networking)** : permet de créer et gérer les réseaux virtuels, sous-réseaux, routeurs et adresses IP.

► 3.2 Environnement de déploiement

Devstack

Utilisé pour :

- Déploiement rapide d'un environnement OpenStack
- Configuration personnalisée via `local.conf`
- Environnement de test et laboratoire

Virtualisation

- Hyperviseur : VirtualBox
- OS : Ubuntu Live-Server 22.04
- RAM VM : 16 Go
- Espace de stockage : 100 Go
- CPU : 6 vCPUs

► 3.3 Technologies

Composant	Technologie
Backend	Python 3.10 + Flask 3.0.0
SDK OpenStack	openstacksdk 1.2.0
Authentification	python-keystoneclient 5.3.0
Frontend	Bootstrap 5.3 + Bootstrap Icons
Visualisation réseau	vis-network.js
Infrastructure réseau	OpenStack Neutron (VXLAN)
Infrastructure compute	OpenStack Nova
Authentification	OpenStack Keystone

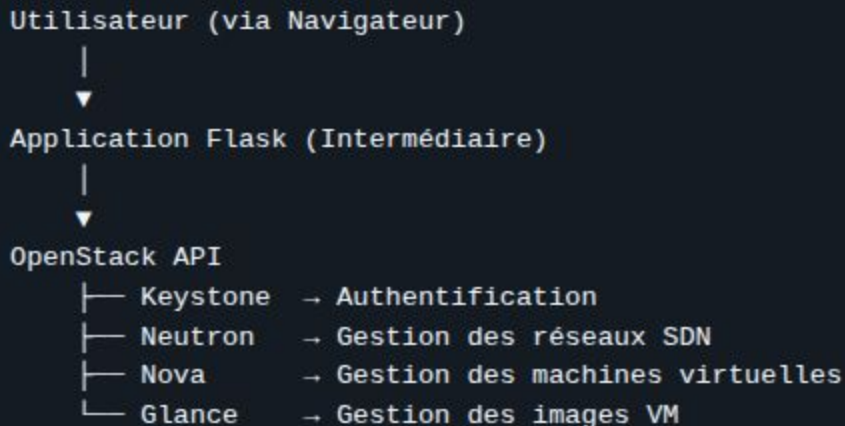
The background is a dark blue gradient with various white and light blue geometric elements. There are thin lines, some forming a grid-like pattern at the top, and others forming abstract shapes. A small circle is connected to a line on the left, and another circle is on the right. A large, faint '04' is visible in the center-right area.

► Architecture Technique

04

► 4.1 Architecture générale

L'application agit comme une couche d'orchestration entre l'interface web et les services OpenStack.



Le backend OpenStack traduit les actions réalisées dans l'interface graphique Flask en appels API vers les services OpenStack.

L'infrastructure réseau repose sur OpenStack Neutron, qui permet de créer des réseaux virtuels isolés entre différents tenants grâce à la technologie VXLAN.

**▶ PHASE DE
RÉALISATION**

05

► 5.1 Mise en place de l'infrastructure

Cette section détaille la mise en œuvre concrète du projet, de l'environnement de développement à l'interface de gestion.

- **Déploiement via DevStack** : Utilisation d'un fichier `local.conf` optimisé pour activer les services Neutron, Nova, Keystone et Glance.
- **Configuration SDN** : Configuration de l'agent ML2 (Modular Layer 2) pour supporter le mécanisme VXLAN, permettant l'isolation de niveau 2 sur un réseau IP existant.

► 5.2 Implémentation du Multi-tenant

- **Scripting Keystone** : Automatisation de la création des projets (Tenant A, B, C) et des utilisateurs associés via l'API.
- **Isolation Réseau** : Attribution d'un ID de segmentation unique pour chaque réseau virtuel de tenant afin d'assurer qu'aucun trafic ne puisse "fuir" d'un environnement à l'autre.

► 5.3 Développement de l'interface de gestion (Flask)

- **Consommation des APIs** : Utilisation de openstacksdk pour mapper les actions de l'interface (clic sur "Créer un routeur") vers les appels API Neutron.
- **Visualisation Dynamique** : Intégration de vis-network.js pour générer un graphe en temps réel de la topologie réseau (VMs, sous-réseaux, routeurs).



▶ CONTRAINTES RENCONTRÉS

06

► 6.1 Contraintes Techniques et Matérielles

Tout projet technique de cette envergure comporte des défis. Voici les points critiques identifiés lors de la réalisation :

- Difficultés de déploiement de la solution Openstack: Difficultés à trouver un guide d'installation fiable pour réussir le déploiement de la solution Openstack.
- Consommation de ressources : OpenStack (même via DevStack) est extrêmement gourmand en RAM et CPU. Le défi a été d'optimiser les quotas pour faire tourner plusieurs tenants simultanément sur une seule machine physique.
- **Gestion des Floating IPs** : Difficulté pour configurer le pont OVS (br-ex) afin de permettre l'accès vers l'extérieur (Internet) depuis les réseaux privés virtuels.

► 6.2 Défis Logiciels

- **Compatibilité des versions** : Ajustement nécessaire entre la version de Flask (3.0.0) et certaines dépendances de openstacksdk pour éviter les conflits de bibliothèques Python.
- **Sécurisation du Plan de Contrôle** : Mise en place de règles de filtrage (Security Groups) assez strictes pour isoler les flux sans bloquer les messages de protocole nécessaires au fonctionnement de VXLAN.

**CONCLUSION ET
PERSPECTIVES**

07

► 7.1 Bilan du projet

Le projet a permis de démontrer qu'il est possible de simplifier l'administration d'une infrastructure complexe comme OpenStack Neutron via une couche d'abstraction (Flask). Les objectifs initiaux de segmentation VXLAN et d'isolation multi-tenant sont atteints et fonctionnels.



► 7.2 Perspectives d'évolution

- **Passage au Multi-node** : Étendre l'infrastructure actuelle (all-in-one) vers une architecture avec plusieurs Compute Nodes pour tester la scalabilité réelle du VXLAN.
- **Monitoring Avancé** : Intégrer un outil comme Prometheus ou Grafana pour surveiller la latence intra-tenant en temps réel.
- **Support IPv6** : Implémenter le support de l'adressage IPv6 pour les réseaux de tenants afin de répondre aux standards modernes de l'industrie.



Merci

pour votre aimable attention !